```java
package com.abhyudayasharma.tictactoe;

import javax.annotation.CheckForNull;
import javax.annotation.Nonnegative;
import javax.annotation.Nonnull;
import javax.annotation.ParametersAreNonnullByDefault;
import javax.swing.BorderFactory;
import javax.swing.BoxLayout;
import javax.swing.JButton;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JList;
import javax.swing.JOptionPane;
import javax.swing.JPanel;
import javax.swing.JScrollPane;
import javax.swing.Timer;
import javax.swing.UIManager;
import javax.swing.UnsupportedLookAndFeelException;
import javax.swing.plaf.nimbus.NimbusLookAndFeel;
import java.awt.Color;
import java.awt.Dimension;
import java.awt.Font;
import java.awt.GridBagConstraints;
import java.awt.GridBagLayout;
import java.awt.GridLayout;
import java.awt.Insets;
import java.util.List;
import java.util.Vector;
import java.util.logging.Level;
import java.util.logging.Logger;

/**
 * This class contains logic for the AI of the game and maintains the UI.
 * The AI is unbeatable.
 *
 * @author Abhyudaya Sharma 1710110019
 * @author Kabir Kanha Arora 1710110165
 */
@ParametersAreNonnullByDefault
class Game {
    /**
     * The current player.
     */
    private PlayerType currentPlayer = null;

    /**
     * The character assigned to the user; either 'X' or 'O'.
     */
    private String userChar;

    /**
     * The character assigned to the computer; either 'X' or 'O'.
     */
    private String compChar;

    /**
     * The size of the tic-tac-toe board = 3.
     */
    private final static int size = 3;

    /**
     * The containing the game.
     */
    private final JFrame gameFrame = new JFrame("Tic Tac Toe");

    /**
     * List of moves made by the user.
     */
    private final Vector<Integer> humanMoves = new Vector<>(9) {
        @Override
        public synchronized boolean add(Integer integer) {
            boolean ret = super.add(integer);
            humanMovesList.setListData(this);
            return ret;
        }
```

```java
    };

    /**
     * List of moves made by the computer.
     */
    private final Vector<Integer> computerMoves = new Vector<>(9) {
        @Override
        public synchronized boolean add(Integer integer) {
            boolean ret = super.add(integer);
            computerMovesList.setListData(this);
            return ret;
        }
    };

    /**
     * Displayable list of moves made by the human user.
     */
    private final JList<Integer> humanMovesList = new JList<>();

    /**
     * Displayable list of moves made by the computer.
     */
    private final JList<Integer> computerMovesList = new JList<>();

    /**
     * Clickable {@link JButton}s for user interactions.
     */
    private JButton[][] buttons = new JButton[size][size];
    private boolean flag = true;

    /**
     * The number of wins made by the human player.
     */
    private static int humanWins = 0;

    /**
     * The number of wins made by the computer.
     */
    private static int computerWins = 0;

    /**
     * JPanel to display the buttons for the game
     */
    private JPanel boxes = new JPanel();

    /**
     * String that stores the name of the current scoreboard leader
     */
    private static String leader = "Nobody";

    /**
     * Magic square used by the AI.
     */
    @Nonnegative
    private final int[][] magicSquare = MagicSquare.getMagicSquare(size);

    /**
     * The sum that gives a user his or her win.
     */
    private final int winningSum = MagicSquare.getExpectedSum();

    // Set up the look and feel to make the UI look good.
    static {
        try {
            //Using Nimbus Look and Feel
            UIManager.setLookAndFeel(new NimbusLookAndFeel());
        } catch (UnsupportedLookAndFeelException e) {
            Logger.getLogger(Game.class.getName()).log(Level.WARNING, "Unable to set Nimbus Look and Feel", e);
        }
    }

    /**
     * Execution starts here.
     */
```

```java
    void start() {
        //Sets the background colour of the main frame
        gameFrame.setBackground(Color.BLACK);

        //Using GridLayout to place buttons
        boxes.setLayout(new GridLayout(3, 3, 5, 5));
        //Setting background colour of the panel
        boxes.setBackground(Color.BLACK);
        //Create an empty border around the panel
        boxes.setBorder(BorderFactory.createEmptyBorder(5, 5, 5, 5));

        /*
         Declares the 9 JButtons
         Adds an action listener to each
         And adds each of them to the panel
         Other visual appearance settings are also handled here
         */
        for (int i = 0; i < size; i++) {
            for (int j = 0; j < size; ++j) {
                buttons[i][j] = new JButton();
                buttons[i][j].setFont(new Font(Font.SANS_SERIF, Font.PLAIN, 44));
                buttons[i][j].setBackground(Color.WHITE);
                buttons[i][j].setPreferredSize(new Dimension(100, 100));
                addOnClickListener(buttons[i][j], i, j);
                boxes.add(buttons[i][j]);
                boxes.setBackground(Color.WHITE);
            }
        }

        //Sets the constraints for using GridBagLayout
        JPanel lists = new JPanel(new GridBagLayout());
        GridBagConstraints gbc = new GridBagConstraints();
        gbc.gridx = 0;
        gbc.gridy = 0;
        gbc.fill = GridBagConstraints.NONE;
        gbc.insets = new Insets(5, 5, 5, 5);
        gbc.anchor = GridBagConstraints.NORTHWEST;
        gbc.weightx = 1;
        gbc.weighty = 0;

        //Adds the label to display the current score on the GUI
        JLabel scoreLabel = new JLabel();
        scoreLabel.setText("SCORE: " + humanWins + " - " + computerWins);
        scoreLabel.setFont(new Font("Arial", Font.BOLD, 14));
        lists.add(scoreLabel, gbc);

        //Displayes the label to show the current leader on the GUI
        gbc.gridy++;
        JLabel leaderLabel = new JLabel();
        leaderLabel.setText("Leader: " + leader);
        leaderLabel.setFont(new Font("Arial", Font.ITALIC, 12));
        lists.add(leaderLabel, gbc);

        //Displays the moves made by the human. Numbering is as per the Magic Square.
        gbc.gridy++;
        lists.add(new JLabel("Human moves"), gbc);
        gbc.gridy++;
        gbc.fill = GridBagConstraints.VERTICAL;
        var humanMovesScrollPane = new JScrollPane(humanMovesList);
        humanMovesScrollPane.setPreferredSize(new Dimension(100, 100));
        lists.add(humanMovesScrollPane, gbc);

        //Displays the moves made by the human. Numbering is as per the Magic Square.
        gbc.gridy++;
        lists.add(new JLabel("Computer moves"), gbc);
        gbc.gridy++;
        var computerMovesScrollPane = new JScrollPane(computerMovesList);
        computerMovesScrollPane.setPreferredSize(new Dimension(100, 100));
        lists.add(computerMovesScrollPane, gbc);
        lists.setBackground(Color.WHITE);

        //Disables user click
        computerMovesList.setEnabled(false);
        humanMovesList.setEnabled(false);

```

```java
        //Sets constraints for the main frame
        gameFrame.setLayout(new BoxLayout(gameFrame.getContentPane(), BoxLayout.LINE_AXIS));
        gameFrame.setResizable(false);
        gameFrame.add(boxes);
        gameFrame.add(lists);
        gameFrame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        gameFrame.setVisible(true);
        gameFrame.pack();
        gameFrame.setLocationRelativeTo(null);

        // Who starts first?
        PlayerType startPlayer = findStartPlayer();
        if (startPlayer == null) {
            gameFrame.dispose();
        }
        currentPlayer = startPlayer;

        //Ensure fist player starts with 'X'
        if (startPlayer == PlayerType.HUMAN) {
            userChar = "X";
            compChar = "O";
        } else {
            userChar = "O";
            compChar = "X";
            computerMove();
        }
        gameFrame.requestFocus();
    }

    /**
     * Adds listener to the button for user interaction which get fired when the button is clicked.
     *
     * @param button the button to which the action listener will be added
     * @param row    zero-indexed row position of the button in the grid
     * @param col    zero-indexed column position of the button in the grid
     */
    private void addOnClickListener(JButton button, int row, int col) {
        button.addActionListener(e -> {
            //flag to check if the game has ended.
            flag = true;
            //Disables button clicked by the user
            button.setEnabled(false);
            gameFrame.requestFocus();

            button.setText(userChar);
            humanMoves.add(magicSquare[row][col]);
            // disable the frame to avoid clicks when we simulate calculation
            gameFrame.setEnabled(false);

            // delay the response of the computer
            Timer timer = new Timer(500, l -> {
                // Check if human has won
                boolean win = humanWin(humanMoves);
                if (win) {
                    gameOver("YOU");
                    flag = false;
                } else {
                    //Check if the grid is full
                    if (humanMoves.size() + computerMoves.size() == 9) {
                        gameOver("NOBODY");
                    }
                    //AI plays now
                    computerMove();
                }
                // re-enable the frame after the computer is done
                gameFrame.setEnabled(true);
            });
            //Prevents the timer from repeating automatically.
            timer.setRepeats(false);
            timer.start();
        });
    }

    /**
```

```java
     * Handles the move of the user.
     */
    private void computerMove() {
        // Win: If you have two in a row, play the third to get three in a row.
        int winEntry;
        if (flag) {
            winEntry = possWin(humanMoves, computerMoves);
            if (winEntry > 0 && winEntry < 10) {
                computerMoves.add(winEntry);
                findAndDisable(winEntry);
                gameOver("COMPUTER");
                flag = false;
            }
        }

        //Block: If the opponent has two in a row, play the third to block them.
        if (flag) {
            winEntry = possWin(computerMoves, humanMoves);
            if (winEntry > 0 && winEntry < 10) {
                computerMoves.add(winEntry);
                findAndDisable(winEntry);
                flag = false;
            }
        }

        //Center: Play the center
        int centre = magicSquare[size / 2][size / 2];
        if (flag) {
            if (!(humanMoves.contains(centre) || computerMoves.contains(centre))) {
                computerMoves.add(centre);
                findAndDisable(centre);
                flag = false;
            }
        }

        /*
         Block Opponent's Forks (Double Attacks)
         */

        //Block centre and corner move by moving in a corner
        if (flag) {
            int[] temp = {2, 4, 6, 8};
            if (humanMoves.size() == 2 && humanMoves.contains(centre) && elementCounter(humanMoves, temp) == 1) {
                if (emptyCorner()) {
                    flag = false;
                }
            }
        }

        //Block two corner move by moving in an empty side
        if (flag) {
            int[] temp = {2, 4, 6, 8};
            if (humanMoves.size() == 2 && elementCounter(humanMoves, temp) == 2) {
                if (emptySide()) {
                    flag = false;
                }
            }
        }

        //Block two side move by moving in corner between them
        if (flag) {
            int[] temp = {1, 3, 7, 9};
            if (humanMoves.size() == 2 && elementCounter(humanMoves, temp) == 2) {
                int move = correctCorner(humanMoves);
                if (move != 0) {
                    computerMoves.add(move);
                    findAndDisable(move);
                    flag = false;
                }
            }
        }

        //Opposite Corner: If the opponent is in the corner, play the opposite corner.
        if (flag) {
```

```java
            if (oppositeCorner()) {
                flag = false;
            }
        }

        //Empty Corner: Play an empty corner.
        if (flag) {
            if (emptyCorner()) {
                flag = false;
            }
        }

        //Empty Side: Play an empty side.
        if (flag) {
            if (emptySide()) {
                flag = false;
            }
        }
        //User's turn
        currentPlayer = getNextPlayer();
    }

    /**
     * Finishes the game and asks the user whether he/she wants to play again.
     *
     * @param winner name of the player who won the game.
     */
    private void gameOver(String winner) {
        for (int i = 0; i < size; ++i) {
            for (int j = 0; j < size; ++j) {
                buttons[i][j].setEnabled(false);
            }
        }
        //Update scoreboard
        if (winner.equalsIgnoreCase("Computer")) {
            computerWins++;
        } else if (winner.equalsIgnoreCase("You")) {
            humanWins++;
        }

        // calculate who's winning right now
        if (humanWins > computerWins)
            leader = "You";
        else if (computerWins > humanWins)
            leader = "Computer";
        else
            leader = "Nobody";
        int result = JOptionPane.showConfirmDialog(gameFrame, "Game Over! " + winner + " WON.\n The score is now " + humanWins + " - " + computerWins + "."
                + "\nCurrent Leader: " + leader + "\nWould you like to play again?", "Game Over",
            JOptionPane.YES_NO_OPTION);
        if (result == JOptionPane.YES_OPTION) {
            gameFrame.dispose();
            new Game().start();
        } else {
            gameFrame.setVisible(true);
        }
    }

    /**
     * Asks the user for input about who starts first.
     *
     * @return the {@link PlayerType} corresponding to the user's selected choice.s
     */
    @CheckForNull
    private PlayerType findStartPlayer() {
        int response = JOptionPane.showOptionDialog(gameFrame, "Welcome to Tic Tac Toe!\n" +
                "Would you like to start first?",
            "Tic Tac Toe", JOptionPane.YES_NO_OPTION, JOptionPane.QUESTION_MESSAGE, null,
            new String[]{"Yes", "No"}, "Yes");

        if (response == JOptionPane.CLOSED_OPTION) {
            return null;
        }
```

```java

        if (response == JOptionPane.YES_OPTION) {
            return PlayerType.HUMAN;
        } else {
            return PlayerType.COMPUTER;
        }
    }

    /**
     * Checks if the winning is possible
     *
     * @param Opponent list of computer moves
     * @param Player   list of human moves
     * @return difference or zero
     */
    private int possWin(List Opponent, List Player) {
        if (Player.size() <= 1)
            return 0;
        int lastMove = (int) Player.get(Player.size() - 1);
        for (int i = 0; i < Player.size() - 1; ++i) {
            int diff = winningSum - ((int) Player.get(i) + lastMove);
            if (!(Opponent.contains(diff) || diff < 1 || diff > 9 || Player.contains(diff))) {
                return diff;
            }
        }
        return 0;
    }

    /**
     * Finds a button from its magic square number and disables it after it has been clicked.
     *
     * @param num the number of the button corresponding to the magic square.
     */
    private void findAndDisable(int num) {
        for (int i = 0; i < size; ++i) {
            for (int j = 0; j < size; ++j) {
                if (magicSquare[i][j] == num) {
                    buttons[i][j].setText(compChar);
                    buttons[i][j].setEnabled(false);
                }
            }
        }
    }

    /**
     * Returns the player who has the next turn.
     *
     * @return the player who has the next turn.
     */
    @Nonnull
    private PlayerType getNextPlayer() {
        switch (currentPlayer) {
            case HUMAN:
                return PlayerType.COMPUTER;
            case COMPUTER:
                return PlayerType.HUMAN;
            default:
                throw new IllegalArgumentException("Unknown PlayerType");
        }
    }

    /**
     * Checks if the win for the human player is possible.
     *
     * @param Player List of player moves.
     * @return true if a win is possible
     */
    private boolean humanWin(List Player) {
        if (Player.size() <= 2)
            return false;
        int lastMove = (int) Player.get(Player.size() - 1);
        for (int i = 0; i < Player.size() - 2; ++i) {
            for (int j = i + 1; j < Player.size() - 1; ++j) {
                int sum = (int) Player.get(i) + (int) Player.get(j) + lastMove;
                if (sum == winningSum)
```

```java
                    return true;
            }
        }
        return false;
    }

    /**
     * Finds the opposite corner for the move.
     *
     * @return opposite corner for the move
     */
    private boolean oppositeCorner() {
        for (int i = humanMoves.size() - 1; i >= 0; --i) {
            if (fillOppositeCorner(humanMoves.get(i))) {
                return true;
            }
        }
        return false;
    }

    /**
     * Finds an empty corner in the matrix
     *
     * @return an empty corner in the matrix
     */
    private boolean emptyCorner() {
        int[] temp1 = new int[]{0, 0, size - 1, size - 1};
        int[] temp2 = new int[]{0, size - 1, 0, size - 1};
        return fillFirstFound(temp1, temp2);
    }

    /**
     * Finds an empty side in the matrix
     *
     * @return an empty side in the matrix
     */
    private boolean emptySide() {
        int[] temp1 = new int[]{0, 1, 1, 2};
        int[] temp2 = new int[]{1, 0, 2, 1};
        return fillFirstFound(temp1, temp2);
    }

    /**
     * Moves in the square identified by two integer arrays.
     * The first corresponding pair location found empty is the move.
     *
     * @param temp1 integer array 1
     * @param temp2 integer array 2
     * @return true if successfully moved
     */
    private boolean fillFirstFound(int[] temp1, int[] temp2) {
        for (int i = 0, j = 0; i < temp1.length && j < temp2.length; ++i, ++j) {
            int num = magicSquare[temp1[i]][temp2[j]];
            if (!(humanMoves.contains(num) || computerMoves.contains(num))) {
                buttons[temp1[i]][temp2[j]].setText(compChar);
                buttons[temp1[i]][temp2[j]].setEnabled(false);
                computerMoves.add(num);
                return true;
            }
        }
        return false;
    }

    /**
     * Moves in a corner opposite to an an already filled one.
     *
     * @param num An entry existing in the list of human moves.
     * @return true is move is successful
     */
    private boolean fillOppositeCorner(int num) {
        for (int i = 0; i < size; ++i) {
            for (int j = 0; j < size; ++j) {
                if (magicSquare[i][j] == num) {
                    if (i == 0 && j == 0) {
                        if (buttons[size - 1][size - 1].isEnabled()) {
```

```java
                                buttons[size - 1][size - 1].setText(compChar);
                                buttons[size - 1][size - 1].setEnabled(false);
                                computerMoves.add(magicSquare[size - 1][size - 1]);
                                return true;
                            }
                        } else if (i == size - 1 && j == 0) {
                            if (buttons[0][size - 1].isEnabled()) {
                                buttons[0][size - 1].setText(compChar);
                                buttons[0][size - 1].setEnabled(false);
                                computerMoves.add(magicSquare[0][size - 1]);
                                return true;
                            }
                        } else if (i == 0 && j == size - 1) {
                            if (buttons[size - 1][0].isEnabled()) {
                                buttons[size - 1][0].setText(compChar);
                                buttons[size - 1][0].setEnabled(false);
                                computerMoves.add(magicSquare[size - 1][0]);
                                return true;
                            }
                        } else if (i == size - 1 && j == size - 1) {
                            if (buttons[0][0].isEnabled()) {
                                buttons[0][0].setText(compChar);
                                buttons[0][0].setEnabled(false);
                                computerMoves.add(magicSquare[0][0]);
                                return true;
                            }
                        }
                    }
                }
            }
        return false;
    }

    /**
     * Counts the number of elements common in an array and a List
     *
     * @param Player the list of player moves
     * @param arr    an integer array
     * @return the number of common elements
     */
    private int elementCounter(List Player, int[] arr) {
        int cnt = 0;
        for (int i1 : arr) {
            if (Player.contains(i1))
                cnt++;
        }
        return cnt;
    }

    /**
     * Chooses the correct corner to move in to block the double side fork
     *
     * @param Player lise of human moves
     * @return the spot for the correct move. 0 if none.
     */
    private int correctCorner(List Player) {
        if (Player.contains(1)) {
            if (Player.contains(3))
                return 8;
            if (Player.contains(7))
                return 6;
        }
        if (Player.contains(9)) {
            if (Player.contains(3))
                return 4;
            if (Player.contains(7))
                return 2;
        }
        return 0;
    }
}
```

```java
package com.abhyudayasharma.tictactoe;

/**
 * Generates a magic square of odd size.
 *
 * @author Kabir Kanha Arora
 */
class MagicSquare {
    private int mainCnt = 0;
    private int[][] matrix;
    private static int expectedSum;

    /**
     * Private constructor for the Magic square.
     *
     * @param n the size of the magic square to be generated.
     */
    private MagicSquare(int n) {
        int max = n * n;
        expectedSum = (int) (n * (Math.pow(n, 2) + 1)) / 2;
        matrix = new int[n][n];
        matrix[0][(n - 1) / 2] = ++mainCnt;
        placeNextOdd(0, (n - 1) / 2, matrix, n, max);
    }

    /**
     * Returns the expected sum of every diagonal, row and column of the Magic Square.
     *
     * @return the expected sum of every diagonal, row and column of the Magic Square.
     */
    static int getExpectedSum() {
        return expectedSum;
    }

    /**
     * Calculates and returns a new magic square object. The objects produced by this methods
     * are isomorphic to one another but are not eht same objects.
     *
     * @param n the size of the magic square to be generated. Even values do not produce expected results.
     * @return a new magic square
     */
    @SuppressWarnings("SameParameterValue") // tic tac toe uses a 3x3 magic square only
    static int[][] getMagicSquare(int n) {
        return new MagicSquare(n).matrix;
    }

    /**
     * Recursive method for placing digits in the magic square.
     *
     * @param r      the row index
     * @param c      the column index
     * @param matrix matrix that will eventually become a magic square
     * @param n      the size of the magic square to be constructed
     * @param max    the maximum possible value in the magic square.
     */
    private void placeNextOdd(int r, int c, int[][] matrix, int n, int max) {
        if (mainCnt == max) {
            return;
        }

        int temp_r, temp_c;

        if (r - 1 < 0) {
            temp_r = n - 1;
        } else {
            temp_r = r - 1;
        }

        if (c + 1 >= n) {
            temp_c = 0;
        } else {
            temp_c = c + 1;
        }

```

```
        if (matrix[temp_r][temp_c] == 0) {
            matrix[temp_r][temp_c] = ++mainCnt;
            placeNextOdd(temp_r, temp_c, matrix, n, max);
        } else {
            if (r + 1 >= n) {
                r = 0;
            } else {
                ++r;
            }
            matrix[r][c] = ++mainCnt;
            placeNextOdd(r, c, matrix, n, max);
        }
    }
}

```

```java
package com.abhyudayasharma.tictactoe;

/**
 * Execution starts here.
 *
 * @author Abhyudaya Sharma
 */
public class Main {
    /**
     * The 'main' method for the game.
     *
     * @param args the command line arguments.
     */
    public static void main(String[] args) {
        new Game().start();
    }
}


```

```java
package com.abhyudayasharma.tictactoe;

/**
 * Utility enum for checking the current player.
 *
 * @author Abhyudaya Sharma
 */
public enum PlayerType {
    HUMAN,
    COMPUTER
}

```